

Problemas 1, 2 e 3 (Trabalho Grafos)

Gustavo Gurgel Medeiros

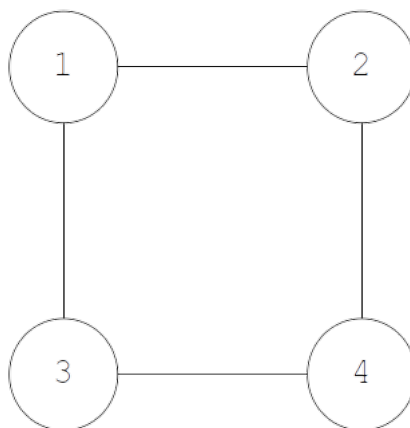
7 de julho de 2023

1 (Porblema 1) (Código em C++) Usando *Duas* Cores, Dizer se Existe Coloração Válida do Grafo.

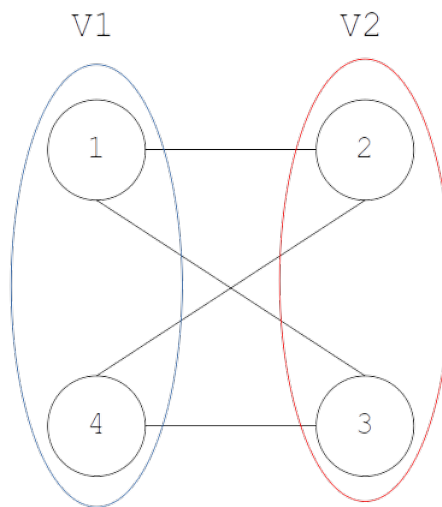
1.1 Interpretação e resolução do Problema

Na questão foi pedido a implementação de um **código em c++** que, dado um grafo $G = (V, E)$ qualquer e usando **duas cores**, retorna se é possível colorir todos os vértices de maneira que vértices vizinhos não possuam a mesma cor. Um pensamento válido para se colorir esse grafo seria fazer uma divisão de seus vértices em 2 *conjuntos disjuntos* V_1 e V_2 , sendo que toda aresta em E tem uma extremidade em V_1 e outra em V_2 . E assim, colorir um conjunto com uma **cor1** e ou outro com uma **cor2**. Como os dois conjuntos são disjuntos e pela nossa definição as arestas não podem ter extremidade no mesmo conjunto, então coloração seria válida. Um grafo que sitisfaz esse propriedade é dito grafo bipartido. Consequentemente, uma grafo tem coloração válida se somente se ele for bipartido.

Fazendo a aplicação desta interpretação para o seguinte grafo com $|V| = 4$:

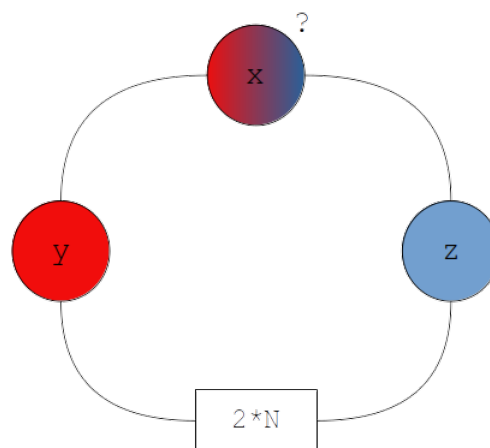


Dividindo o grafo da maneira que foi proposto chegamos ao seguinte resultado:



Como pode ser visto o grafo foi devidamente pintado. Logo, para dizer se é possível pintar o grafo com duas cores, basta descobrir se ele é bipartido ou não. Para isso temos que achar ciclos de tamanho ímpar. Se um grafo não tem ciclo de tamanho ímpar, então ele é bipartido. Isso acontece pois sempre que temos um ciclo de tamanho ímpar, temos um vértice que conecta duas extremidades de um caminho par. Como para todo caminho par suas extremidades vão ter cores diferentes, qualquer que seja a cor do vértice que conecta essas duas extremidades vai violar nossa propriedade. Isso fica bem ilustrado na imagem abaixo:

Ciclo de tamanho ímpar



Portanto, finalizando o raciocínio, é preciso desenvolver um algoritmo que encontre ciclos de tamanho ímpar. Se algum caminho de tamanho ímpar for encontrado então o grafo **não pode ser bipartido** e consequentemente **não existe coloração válida**. Caso o contrário, se nenhum ciclo de tamanho ímpar existir, então o grafo é bipartido, assim, **existe uma coloração válida**.

1.2 Explicação do algoritmo

Para essa questão foi desenvolvida uma classe `Graph`, que representa um grafo utilizando listas de adjacências. Como a classe é simples, não vejo necessidade de documentar todas as suas propriedades aqui. Dessa forma, vamos focar no método principal que resolve o problema passado pela questão. Esse método é o `canBeColored()` que retoma um valor booleano para se existe ou não coloração válida.

Para achar ciclos de tamanho ímpar, a função `canBeColored()` utiliza **busca por profundidade**. Dois vetores são usados, um para armazenar as **cores** de cada vértice, e outro para armazenar a **distância percorrida** da origem até o vértice em questão, sendo que a origem tem distância 0 dela mesma. Assim, após os vetores de cores e distâncias serem inicializados com os valores branco e 0 respectivamente, um loop é iniciado para definir uma origem (um vértice branco) a ser explorada. Depois que a origem u é definida, um loop de exploração começa a fazer a busca por profundidade em todos os vértices conexos a u . Se em algum momento da busca um vértice cinza possuir um outro vértice cinza adjacente a ele, então é porque temos um ciclo no grafo. Calculamos a distância absoluta entre esses dois vértices e isso retoma a quantidade de aresta entre os dois. Essa distância é somada $+1$, pois existe uma aresta a mais que conclui o ciclo. Se essa ao final da conta essa distância for um **valor maior que dois e ímpar**, então achamos um ciclo de tamanho ímpar. Consequentemente o grafo não pode ser bipartido, logo a função retorna **false**. Se após exploração completa do grafo não forem encontrados ciclos de tamanho ímpar, então o **grafo é bipartido** e consequente mente existe coloração válida, assim a função retorna **true**. Assim, finalizamos a explicação do algoritmo implementado.

1.3 Análise de complexidade de tempo e espaço

Para esse algoritmo, o pior caso é quando **existe uma coloração valida**, pois neste caso a busca por profundidade vai rodar até o final, até que não se ache ciclos de tamanho ímpar. Com isso, como já foi discutido em sala, para a busca em profundida sua complexidade de tempo é da ordem de $\mathcal{O}(|V| + |E|)$. Além disso, como estamos representando o grafo como uma lista de adjacência, o espaço utilizado é da também é da ordem de $\mathcal{O}(|V| + |E|)$.

2 (Porblema 2) (Código em C++) Calcular "Número de Bacon" de atores

2.1 Interpretação e resolução do Problema

A interpretação desse problema é de certa maneira simples. Dado um grafo de atores que representa as relações entre atores que fizeram os mesmos filmes, precisamos encontrar o **número de bacon** de todos os atores. O número de bacon é representado pelo **menor caminho entre Kevin Bacon e um vértice qualquer do grafo**. Quando pensamos em

menor caminho, busca em largura parece ser uma alternativa viável. Isso porque dado uma origem u qualquer e um vértice v . Se existir uma caminho entre u e v , então a busca em largura vai retorna a caminho mínimo entre u e v . Como esse teorema esse já foi provado em sala e nos slides, não vou refazer a prova aqui. Sabendo disso, uma resolução para esse algoritmo seria criar o grafo de relações de Atores e chamar a busca em largura tendo Kevin Bacon como origem. Dessa forma, Kevin Bacon tem número de bacon 0 e todo vértice adjacente a ele tem número de bacon = $0 + 1$ e vértices adjacentes a vértices adjacentes a Kevin Bacon tem número de bacon = $1 + 1$ e assim por diante. Após isso, armazenamos todos esses valores em uma tabela e fazemos a impressão.

2.2 Explicação do algoritmo

Para mim, o desafio principal dessa questão não foi calcular o número de bacon, mas sim representa o grafo de Atores. Até então, cada grafo era representado por um valor numérico. Dessa forma era possível fazer a lista de adjacência apenas como um vetor de acesso direto. Porém, agora como cada vértice do grafo seria representado por um ator, foi necessário adaptar a antiga implementação. Assim, para contorna a não possibilidade de acesso direto, mas ainda efetuar inserções e requisições em média de tempo $\mathcal{O}(1)$, a lista de adjacência foi implementada como uma `HashTable<K,T>`, sendo que K é uma string representado o **nome do ator** e T sendo uma lista de **structs do tipo Actor**. Veja a implementação abaixo:

```
//Estrutura que representa uma ator
struct Actor
{
    string name;
    string film;
};

class Graph{
private:
    uint _E;           //Número de arestas
    uint _V;           //Número de vértices

    HashTable<string, list<Actor>> _adj; //Lista de adjacências
    AvlTree<string> _VTree;             //Ávore com os vértices
```

Perceba que necessidade da struct Actor surge pois o vértice que conecta dois atores tem uma informação extra, que no caso é o nome do filme que os dois participaram. Essa informação precisa ser guardada pois problema pede que além do número de bacon do ator, seja passado o filme pelo qual ele recebe esse número. Além disso, como os vértices não pertencem a um intervalo inteiro, é preciso guardar os vértices que foram adicionados no grafo de forma que não tenham vértices duplicados. Com isso, pensando em uma estrutura de dados que não permita elementos duplicados e inserções em tempo logarítmico, eu decidi utilizar uma árvore AVL. Essa estrutura ajuda ainda mais, pois pelo fato de ela ser uma árvore binária de busca, pode-mos usar inorder para pegar os elementos ordenados em tempo linear. Isso casa perfeitamente com o fato da saída do programa ser o número de bacon dos vértices em ordem alfabética.

Para adicionar elementos no grafo eu fiz uma função chamada `halfInsert(u, v)`, que dados dois atores u e v a função adiciona apenas a aresta unidirecional de $u \rightarrow v$. Assim, na função

pública `insert(u,v)`, `halfInsert` é chamado duas vezes adicionado as arestas $u \rightarrow v$ e $v \rightarrow u$. Isso pode ser analisado na implementação abaixo:

Função privada <pre> void halfInsert(Actor u, Actor v){ //O grafo ainda não possui o vértice u if(!_adj.containsKey(u.name)){ //Lista é criada _adj.add(u.name, list<Actor> {v}); //Vértice é adicionado a árvore de vértices _VTree.add(u.name); _V++; } //O grafo já tem o vértice u else{ //Aresta é adicionada na lista de adj _adj.at(u.name).push_back(v); } } </pre>	Função pública <pre> //Adicina uma aresta de forma completa void insert(Actor u, Actor v){ halfInsert(u, v); halfInsert(v, u); _E++; //Aresta adicionada } </pre>
---	---

Perceba que função `halfInsert(u, v)` analisa se na HashTable já existe a lista de adjacência com a **key** = **nome_ator**. Se sim, então ele só adiciona na lista já existente, caso o contrário a lista para aquela chave é criada e o vértice é adicionado a árvore.

Agora, vamos falar da função `getBaconTable`. Essa função é a que de fato resolve o problema em questão. Veja o início da sua declaração abaixo:

```

void getBaconTable(HashTable<string, pair<uint, string>>& ref_table){
    HashTable<string, char> colors(_V); // w = branco; g = cinza; b = preto
    vector<string> V;                    // vetor de vértices
    queue<string> Q;                     // fila de exploração

    _VTree.inoderVec(V);                 // pega todos os vértices

    //Toda os vértices ficam brancos
    for(string str : V){
        colors.add(str, 'w');
    }

    //A origem da nossa busca é o Kevin Bacon
    Q.push("Kevin Bacon");
    //Kevin Bacon tem número de bacon = 0
    ref_table.add("Kevin Bacon", make_pair(0, ""));
}
                
```

Essa função recebe uma referência a uma HashTable do tipo `<string, pair<uint, string>>` e vai adicionar elementos de tal forma que no final da função todo vértice (nome de ator) do grafo seja uma chave ligada a um par de elementos, sendo primeiro o número de bacon e o segundo o nome do filme pelo qual o vértice recebe esse número. Além disso, a função cria uma HashTable auxiliar que vai representar as cores de cada vértice na tabela. Após isso, Kevin Bacon é adicionando em uma queue para ser a origem da nossa busca. Como é possível ver abaixo, a busca por largura se inicia e adicionamos na tabela os pares relacionados a cada ator:

```

while(!Q.empty()){
    string u = Q.front();
    Q.pop();

    // Exploramos todos os vértices adjacentes
    for(Actor& ac : _adj.at(u)){
        //Vértice não explorado
        if(colors.at(ac.name) == 'w'){
            //pintamos ele de cinza
            colors.add(ac.name, 'g');
            //Adicionamos seu número de bacon e o filme
            //Lembrando que:
            //Número de banco = (número de bacon origem) + 1
            ref_table.add(ac.name, make_pair(ref_table.at(u).first+1, ac.film));
            //Adicionamos vértice na fila
            Q.push(ac.name);
        }
    }

    //se chegou aqui é porque terminamos
    //de explorar u. Então pintar de preto
    colors.add(u, 'b');
}
    
```

Ao final, a tabela passado como referência vai conter para todo nome de ator, seu número de bacon e filme que faz ele receber esse número. Após isso, o resto da implementação é apenas leitura e escrita de dados na main, que não vou debater aqui. Concludentemente, a explicação da implementação é para esse problema é finalizada.

2.3 Análise de complexidade de tempo e espaço

Para esse algoritmo, creio que a função principal é a `getBaconTable`, considerando apenas a busca em largura para essa função e levando em conta que os acessos a tabela de cores é em média $\mathcal{O}(1)$, então como já foi debatido em sala e nos slides, a complexidade para esse busca é da ordem de $\mathcal{O}(|V| + |E|)$. Em questão de utilização espaço, a lista de adjacência usa $\mathcal{O}(|V| + |E|)$ de espaço e a árvore AVL que guarda todos os vértices $\mathcal{O}(|V|)$.

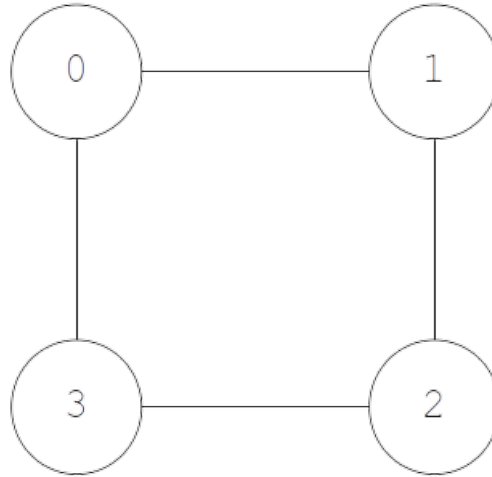
3 (Porblema 3) (Código em C++) Converter ruas de mão dupla em mão única com caminho para qualquer cidade

3.1 Interpretação e resolução do Problema

Nessa questão foi pedido que, dado um grafo não direcionado que representa as ruas de mão dupla de uma cidade, fosse convertido todas as ocorrências possíveis de vias de mão dupla em mão única. Ou seja, o que devemos fazer é pegar um grafo não direcionado e de alguma forma torna para maioria dos vértices o grau de entrada e saída em 1. Pensando nisso, a primeira ideia que eu tive foi "caminhar/explorar" de alguma forma o grafo e fazer ligações entre os vértices. Assim, uma das alternativas seria fazer busca por largura ou profundidade e criar um novo grafo para guardar o resultado dessa busca. Porém, fica a dúvida, *qual das duas buscas é melhor para o nosso problema?*. Vamo fazer uma uma comparação para seguinte grafo:

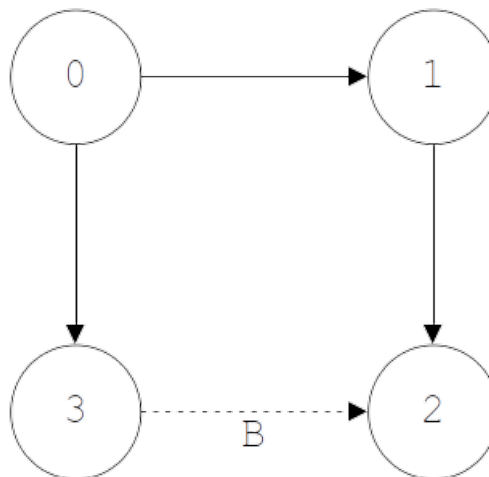
Grafo

$$|V| = 4 \quad |E| = 4$$



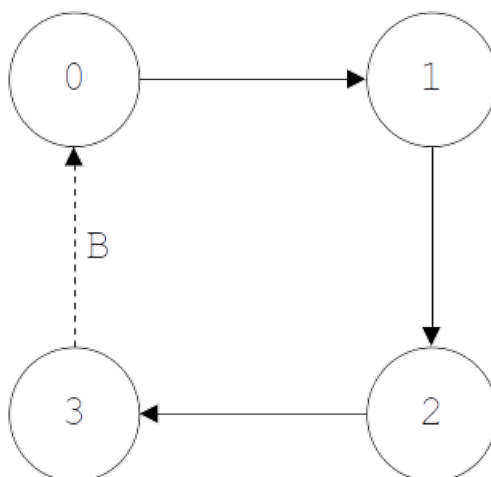
Um resultado válido para a busca em largura tendo como origem o vértice 0:

Grafo Busca em Largura



Um resultado válido para a busca em profundidade tendo como origem o vértice 0:

Grafo Busca em profundidade

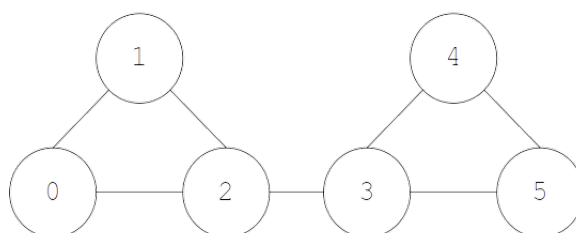


Perceba que busca em profundidade produz um caminho mais linear que a busca em largura. Além disso, se consideramos os backedges podemos criar grafos com ciclos de pistas únicas, o que encaixa perfeitamente na questão.

Com isso, chegamos que uma possível solução para o problema. Porém, perceba que ainda existe uma complicação a ser resolvida. Para muitos grafos, quando adotamos essa metodologia, a conversão acaba criando componentes fortemente conexas, ou seja, subgrupos de vértices que tem caminho para todos elementos dentro grupo, mas não para elementos fora do grupo.

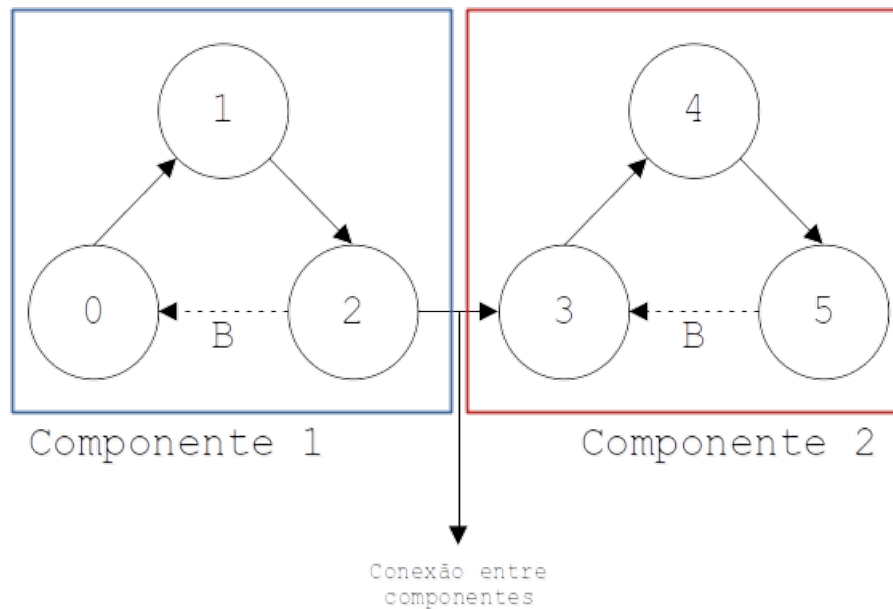
Um exemplo bem claro disso pode ser visto se metodologia for aplicada para o seguinte grafo:

Grafo $|V| = 6$ e $|E| = 7$



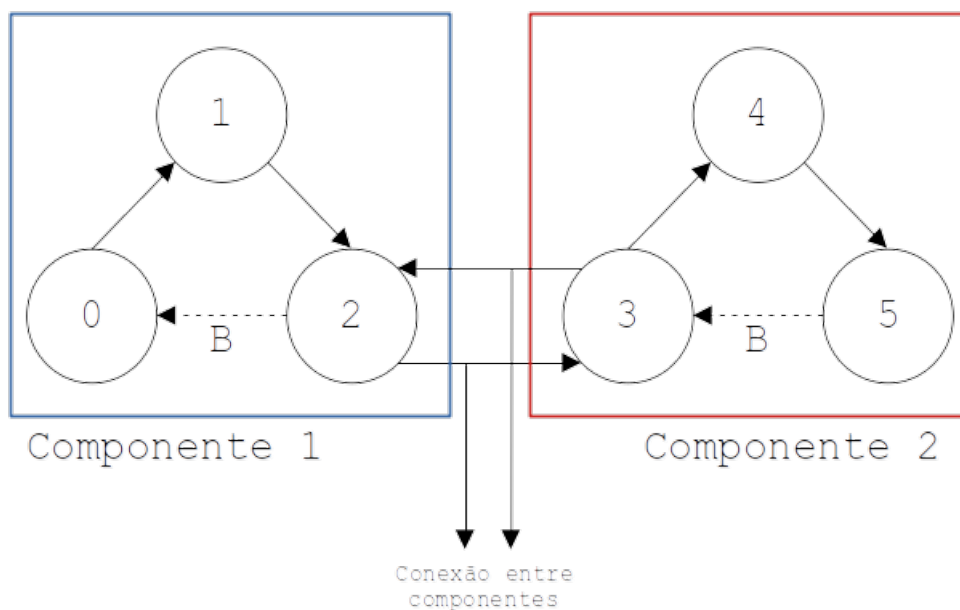
Depois da conversão temos a seguinte análise:

Grafo Depois Da Conversão



Perceba que as componentes têm caminhos entre todos seus vértices, porém temos apenas um caminho de ida da componente 1 a 2. Por isso para todas as componentes precisamos fazer conexão de ida e volta. Ficando com o seguinte grafo

Grafo Depois Da Conversão



Depois de conectar as componentes obtemos o gráfico da maneira que a questão pede. Assim, concluímos o desenvolvimento da solução do problema.

3.2 Explicação do algoritmo

Nessa questão vou explicar o algoritmo descrevendo os métodos principais da classe grafo que resolve o problema. Isso porque eu acho que o procedimento para resolver o problema já foi explicado muito bem no tópico anterior:

1. `malformedConvert()`

- Usa busca por profundidade para criar o grafo direcionado. Considerando backedges para criar ciclos e componentes
- Essa função faz uma espécie de conversão "malformada" do grafo, sem conexões entre as componentes.
- É uma função privada pois é apenas de uso interno da classe graph.

2. `getComponents()`

- Retorna um vetor com as componentes de um grafo direcionado
- usa o Kosaraju's Algorithm (tempo $\mathcal{O}(|V| + |E|)$)

3. `convertInDirected()`

- Função principal que resolve o problema da questão
- Essa função usa ambas as funções citadas anteriormente. Primeiramente ele pega a forma malformada do grafo que chamou o método. Depois usa função `getComponents` para pegar as componentes do grafo malformado. Por fim, faz uma busca em profundidade pelo grafo e conserta todas as arestas que ligam componentes do grafo, da mesma forma que foi debatido no tópico de interpretação dessa questão.

3.3 Análise de complexidade

Nessa questão as funções principais são a `malformedConvert()`, `getComponents()` e `convertInDirected()`. Como já foi dito no tópico anterior a função `getComponents()` usa o **Kosaraju's Algorithm** que internamente usa busca em profundidade para encontrar as componentes, ou seja, sua complexidade é $\mathcal{O}(|V| + |E|)$. Da mesma forma, a função `malformedConvert()` usa busca em largura para converter um grafo não direcionado em direcionado, assim, sua complexidade também é $\mathcal{O}(|V| + |E|)$. E por último a função `convertInDirected()` usa os dois métodos anteriores, além de fazer mais uma busca por largura para consertar as arestas que conectam componentes, lembrando que como já foi debatido em sala e nos slides, a busca em largura tem complexidade $\mathcal{O}(|V| + |E|)$. Finalmente, ao juntar todos esses métodos, temos um algoritmo que ao final tem complexidade de $\mathcal{O}(|V| + |E|)$. Assim, finalizamos o raciocínio da questão.